GAME OVERLAY & HUD PLUGIN .deckxoverlay / .dxoverlay

DeckX Game Overlay & Telemetry HUD Developer Guide

Specification & Architecture for In-Game Telemetry, Hardware HUDs & Settings

Interface: `IDeckXGameOverlay`

Target Hardware: DeckX Control Pad ESP32 (480×272 RGB565) & DX8K (800×480)

NuGet Package: `DeckX.Core` v1.0.0

1. Architecture Overview

DeckX Game Overlay Plugins allow developers to create custom in-game monitoring HUDs and hardware telemetry displays (FPS, CPU, GPU, RAM, VRAM) that render both on the physical DeckX Control Pad ESP32 display and in the DeckX Studio live preview.

Each overlay is authored as a **single C# Class Library project** targeting .NET 8.0, referencing the official `DeckX.Core` NuGet package. The project bundles:

- The compiled assembly DLL implementing `IDeckXGameOverlay`
- A built-in custom settings UI (Windows Forms or custom dialog)
- `overlay.json` manifest and `preview.png` gallery thumbnail
- `settings.json` defining default color schemes and sensor label overrides

> ⚡ **1-Click Build & Auto-Deploy:**
> Running `dotnet build -c Release` compiles your code, packs everything into a `.deckxoverlay` zip bundle, and automatically deploys it into DeckX Studio's AppData directory (`%APPDATA%\DeckX\devices\control_pad\dx8k\game_overlays\<id>\`). You can immediately test it live on the physical device!

2. NuGet Package Installation & Project Configuration

Create a new .NET 8.0 Windows Class Library project and install the official `DeckX.Core` NuGet package:

```
dotnet new classlib -n MyGameOverlay -f net8.0-windows
dotnet add package DeckX.Core --version 1.0.0
dotnet add package SkiaSharp --version 2.88.9
```

Recommended `.csproj` Configuration

Ensure `DeckX.Core` has `PrivateAssets="all"` and `ExcludeAssets="runtime"` so that the host application's DLL is not duplicated in your plugin package:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0-windows</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <UseWindowsForms>true</UseWindowsForms>
    <CopyLocalLockFileAssemblies>true</CopyLocalLockFileAssemblies>
  </PropertyGroup>

  <!-- Fallback to official NuGet package for standalone developer builds -->
  <ItemGroup Condition="!Exists('..\..\DeckX.Core\DeckX.Core.csproj')">
    <PackageReference Include="DeckX.Core" Version="1.0.0">
      <PrivateAssets>all</PrivateAssets>
      <ExcludeAssets>runtime</ExcludeAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="SkiaSharp" Version="2.88.9" />
  </ItemGroup>

</Project>
```

3. The `IDeckXGameOverlay` Interface

Defined in `DeckX.Core.Plugins`, this contract gives you full control over the overlay identity, settings dialog, and live SkiaSharp 2D rendering:

```
using System;
using DeckX.Core.Models;
using SkiaSharp;

namespace DeckX.Core.Plugins;

public interface IDeckXGameOverlay
{
    // Unique ID (e.g. "com.developer.cyber_hud")
    string Id { get; }

    // Human-readable display name shown in the gallery
    string Name { get; }

    // Author / Studio name
    string Author { get; }

    // Semantic version string (e.g. "1.0.0")
    string Version { get; }

    // Short description of features and layout
    string Description { get; }

    // If true, displays the Settings (⚙) button on the gallery card
    bool HasSettings => true;

    // Called when the user clicks the Settings (⚙) button
    void OpenSettings();

    // Event raised when settings are saved to trigger immediate screen refresh
    event Action? SettingsSaved;

    // Called on load with the absolute path to your package directory
    void OnInitialize(string packageDirectory) { }

    // Real-time SkiaSharp rendering callback (480x272 RGB565)
    void RenderOverlay(SKCanvas canvas, int width, int height, GameOverlayTelemetryData telemetry);
}
```

4. Telemetry Data Model (`GameOverlayTelemetryData`)

DeckX Studio continuously collects hardware statistics from LibreHardwareMonitor and in-game hooks, passing them into your `RenderOverlay` callback:

Property	Type	Description
`ActiveGameName`	`string`	Name of the currently running foreground game or process (e.g., "Cyberpunk 2077").
`Fps`	`int`	Real-time framerate reported by the graphics hook (0 if idle on desktop).
`CpuUsagePercent`	`int`	Overall CPU utilization (0 – 100%).
`CpuTempC`	`int`	CPU package temperature in degrees Celsius.
`CpuClockGhz`	`double`	Current CPU core clock frequency in GHz.
`GpuUsagePercent`	`int`	Primary GPU core load (0 – 100%).
`GpuTempC`	`int`	GPU core junction temperature in degrees Celsius.
`RamUsedGb` / `RamFreeGb`	`double`	System physical memory usage in Gigabytes.
`RamUsagePercent`	`int`	System RAM utilization percentage (0 – 100%).
`GpuVramUsedGb` / `GpuVramTotalGb`	`double`	Dedicated Video RAM usage in Gigabytes.

5. Package Structure & `overlay.json` Manifest

An overlay package is a standard ZIP archive with the extension `.deckxoverlay` containing the following files:

```
MyGameOverlay.deckxoverlay/
 ├── MyGameOverlay.dll       <-- Compiled assembly containing IDeckXGameOverlay
 ├── overlay.json            <-- Package manifest metadata
 ├── settings.json           <-- Configurable colors & sensor label overrides
 ├── preview.png             <-- Thumbnail for DeckX Studio gallery (480x272 or 16:9)
 └── assets/                 <-- Optional custom fonts or sprites
```

`overlay.json` Manifest Example:

```
{
  "$schema": "https://deckx.app/schemas/game-overlay.schema.json",
  "id": "com.deckx.sample.gameoverlay",
  "name": "Cyber Neon Telemetry HUD",
  "author": "DeckX Developer Community",
  "version": "1.0.0",
  "assembly": "DeckX.GameOverlay.SampleTemplate.dll",
  "preview": "preview.png",
  "settings": true,
  "layout": "custom",
  "targetDevice": "DX8K",
  "resolution": {
    "width": 480,
    "height": 272
  }
}
```

6. Settings Dialog & Two-Way Synchronization

When the user clicks the **Settings (⚙)** button on the overlay card in DeckX Studio, DeckX invokes your `OpenSettings()` method.

```
User clicks [Settings ⚙] on Overlay Card in DeckX Studio │ ▼ DeckX invokes: overlay.OpenSettings() │ ▼ Your DLL launches SettingsWindow (STA Thread): ├── Loads current settings.json from package directory ├── Provides color pickers (Background, Accent, Card, Text) ├── Provides hardware name overrides (Custom CPU/GPU text) & metric toggles │ ▼ User clicks [Save & Apply] ├── Saves updated values to settings.json └── Fires: SettingsSaved?.Invoke() │ ▼ DeckX Studio catches event ➔ instantly flushes color cache and updates physical ESP32 screen!
```

Implementing `OpenSettings` and `SettingsSaved`:

```
public bool HasSettings => true;
public event Action? SettingsSaved;

public void OpenSettings()
{
    if (string.IsNullOrEmpty(_packageDirectory)) return;

    var thread = new Thread(() =>
    {
        try
        {
            var result = SettingsWindow.ShowDialog(_packageDirectory);
            RefreshSettingsCache();
            if (result == DialogResult.OK)
            {
                // Notify DeckX Studio to sync changes to physical ESP32 and Studio UI
                SettingsSaved?.Invoke();
            }
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine($"[Overlay] Error showing settings: {ex.Message}");
        }
    });

    thread.SetApartmentState(ApartmentState.STA);
    thread.IsBackground = true;
    thread.Start();
}
```

7. Real-time SkiaSharp 2D HUD Rendering (480×272)

The DeckX Control Pad ESP32 runs a 480×272 RGB565 screen. Your `RenderOverlay` method receives a pre-configured `SKCanvas` surface ready for 2D drawing.

Example: Drawing Status Bar, Thermal Gauges & Memory Bars

```
public void RenderOverlay(SKCanvas canvas, int width, int height, GameOverlayTelemetryData telemetry)
{
    // 1. Clear background
    SKColor bgColor = SKColor.Parse(_cachedBgHex);
    canvas.Clear(bgColor);

    // 2. Draw Top Bar with Active Game Name and Framerate
    using (var titlePaint = new SKPaint { Color = SKColors.Cyan, TextSize = 13, IsAntialias = true, Fa
    using (var fpsPaint = new SKPaint { Color = SKColors.White, TextSize = 15, IsAntialias = true, Fak
    {
        string gameTitle = !string.IsNullOrWhiteSpace(telemetry.ActiveGameName) ? telemetry.ActiveGame
        canvas.DrawText(gameTitle, 12, 19, titlePaint);
        canvas.DrawText($"{telemetry.Fps} FPS", width - 12, 19, fpsPaint);
    }

    // 3. Draw CPU & GPU Cards (Top Half)
    float cardW = 231, cardH = 144;
    DrawTelemetryCard(canvas, 6, 28, cardW, cardH, "CPU", telemetry.CpuUsagePercent, telemetry.CpuTemp
    DrawTelemetryCard(canvas, 243, 28, cardW, cardH, "GPU", telemetry.GpuUsagePercent, telemetry.GpuTe

    // 4. Draw RAM & VRAM Memory Cards (Bottom Half)
    DrawRamCard(canvas, 6, 176, cardW, 90, telemetry);
    DrawVramCard(canvas, 243, 176, cardW, 90, telemetry);
}

private void DrawTelemetryCard(SKCanvas canvas, float x, float y, float w, float h, string label, int
{
    // Draw background card with rounded borders
    using var bg = new SKPaint { Color = new SKColor(16, 19, 29), Style = SKPaintStyle.Fill, IsAntiali
    using var border = new SKPaint { Color = new SKColor(35, 41, 59), Style = SKPaintStyle.Stroke, Str
    canvas.DrawRoundRect(new SKRoundRect(new SKRect(x, y, x + w, y + h), 8, 8), bg);
    canvas.DrawRoundRect(new SKRoundRect(new SKRect(x, y, x + w, y + h), 8, 8), border);

    // Label & Temp
    using var textPaint = new SKPaint { Color = SKColors.White, TextSize = 11, IsAntialias = true };
    canvas.DrawText(label, x + 10, y + 20, textPaint);
    canvas.DrawText($"{temp}°C", x + w - 10, y + 20, textPaint);

    // Large Percentage Readout
    using var valPaint = new SKPaint { Color = SKColors.White, TextSize = 34, FakeBoldText = true, IsA
    canvas.DrawText($"{usage}%", x + 10, y + 68, valPaint);

    // Progress bar fill
    float barW = w - 20, barH = 6;
    using var fill = new SKPaint { Color = accent, Style = SKPaintStyle.Fill, IsAntialias = true };
    canvas.DrawRect(x + 10, y + 80, barW * (usage / 100f), barH, fill);
}
```

8. Summary & Best Practices

- **Keep Allocations Minimal:** `RenderOverlay` is called up to 10–20 times per second during gaming sessions. Cache `SKPaint` instances or reuse brush objects to avoid garbage collection pressure.

- **Cache Settings Reads:** Avoid reading `settings.json` from disk on every render frame. Cache parsed values in memory and refresh only when `SettingsSaved` is triggered or every couple of seconds.
- **Always Fire `SettingsSaved`:** After the settings window writes changes to `settings.json`, raise `SettingsSaved?.Invoke()` so DeckX Studio flushes serial buffers and refreshes the hardware screen instantly.